

Detección de Clientes: Suscripción de Depósito Bancario

Facundo Casas

facundocasas@uca.edu.ar

Universidad Católica Argentina

Javier Balda

javierbalda@uca.edu.ar

Universidad Católica Argentina

Juan Caracoix

juancaracoix@uca.edu.ar

Universidad Católica Argentina

ABSTRACT

La explicabilidad es un aspecto fundamental del aprendizaje automático, necesario para garantizar transparencia, interpretabilidad y confianza en los procesos de toma de decisiones. [...]

Palabras clave: Inteligencia Artificial, Modelos de Caja Negra, Similitud del Coseno, Aprendizaje Profundo, Explicabilidad, Redes Neuronales, Lógica, Regularización, Extracción de Reglas

1. INTRODUCCIÓN

Las redes neuronales profundas (DNN) han demostrado tener alta precisión predictiva y un rendimiento superior en diversos problemas de aprendizaje automático. [...]

2. EXPLICABILIDAD

La explicabilidad proporciona las bases para interpretar cómo los modelos generan salidas y comprender el razonamiento detrás de sus predicciones. [...]

3. COLOSSUS

3.1. Análisis de Contexto

El estudio examina cómo la similitud del coseno puede utilizarse para analizar los pesos y sesgos neuronales. [...]

3.2. Alcance

El marco COLOSSUS se centra en redes neuronales feedforward con capas completamente conectadas. [...]

3.3. Método Propuesto

El método propuesto integra similitud del coseno y lógica de primer orden (FOL) para generar reglas interpretables. [...]

4. IMPLEMENTACIÓN Y DESPLIEGUE

En esta sección se documentan los componentes de software desarrollados para entrenar, servir y procesar los datos del modelo, basados en la implementación disponible en la carpeta `scr/` del repositorio.

4.1. Modelo y artefactos

El modelo entrenado se empaqueta como un pipeline serializado en `scr/app/model/trained_pipeline-` y su versión explícita en el código es 0.1.0. La interfaz de inferencia expone dos funciones principales en `scr.app.model.model`:

- `predict_pipeline(input_data)`: devuelve la clase predicha (0/1).
- `predict_pipeline_proba(input_data)`: devuelve la clase y las probabilidades por clase.

La lista de variables esperadas por el pipeline (`FEATURE_COLUMNS`) incluye columnas numéricas y variables dummificadas. Es crítico que los datos de entrada respeten este orden y la presencia de columnas; en caso de discrepancia el servicio devuelve errores indicando las columnas faltantes.

4.2. API y Endpoints

La aplicación web se implementa con FastAPI en `scr/app/main.py` y en `scr/app/routes`. Los endpoints relevantes documentados en el código son:

- `GET /`: información básica y versión del modelo.

- GET `/ping`: comprobación simple (pong).
- GET `/healthz`: health check.
- GET `/readyz`: readiness (verifica si el modelo está disponible).
- GET `/info`: información del servicio y entorno.
- POST `/predict` y POST `/predict/single`: endpoints de inferencia que aceptan el esquema `ClientData` (Pydantic) y devuelven la predicción y probabilidades.

El código incluye manejo de errores centralizado mediante el decorador `handle_exceptions` definido en `scr/utils/errors.py`, que traduce excepciones internas a respuestas HTTP apropiadas.

4.3. Pipeline de Entrenamiento

El flujo de entrenamiento está definido en `scr/training/train_pipeline.py` y utiliza Prefect para orquestación. Los pasos principales son:

- Carga de datos desde una base MySQL mediante SQLAlchemy y guardado temporal en parquet.
- Aplicación de oversampling (SMOTE) para balancear la variable objetivo (en la implementación se fuerza un mínimo por clase, por defecto se apunta a 10.000 ejemplos para la clase minoritaria en algunos pasos).
- Guardado de los datos transformados en una tabla MySQL preparada (esquema definido en `scr/training/esquema_DB_train.py`).
- Limpieza de archivos temporales en `/tmp/bancox_train`.

El uso de SMOTE y el número objetivo por clase son decisiones de diseño que deben ser justificadas y validadas experimentalmente.

4.4. Procesamiento y Preprocesado

El módulo de procesamiento principal es una clase abstracta `Processor` definida en `scr/processing/processor.py`, con una implementación de ejemplo `ETLProcessor` que realiza validaciones básicas (comprobación de columnas esperadas), registro de columnas faltantes y limpieza ligera (eliminación de duplicados). Esta abstracción facilita crear pasos de ETL reproducibles y con logging estandarizado.

4.5. Registro y Monitoreo

El proyecto incluye un wrapper de logging en `scr/utils/log.py` que configura logging con un `RotatingFileHandler` y soporte para configurar fichero y nivel mediante variables de entorno (`LOG_FILE`, `LOG_LEVEL`). Prefect se utiliza para telemetría adicional en flujos y tareas asíncronas.

4.6. Requisitos y Ejecución

El servicio web contiene un `requirements.txt` en `scr/app/` que lista dependencias de la API. El pipeline de entrenamiento depende de paquetes adicionales como `pymysql`, `sqlalchemy`, `imblearn` y `prefect`. Para ejecutar localmente:

- Iniciar la API con Uvicorn apuntando a `scr.app.main:app`.
- Ejecutar el flow de entrenamiento mediante Prefect o ejecutando directamente `scr/training/train_` (requiere variables de entorno para MySQL).

5. ARQUITECTURA DEL SISTEMA

En la Figura ?? se muestra el diagrama arquitectónico que resume los componentes principales del sistema, las relaciones entre ellos y los flujos de datos.

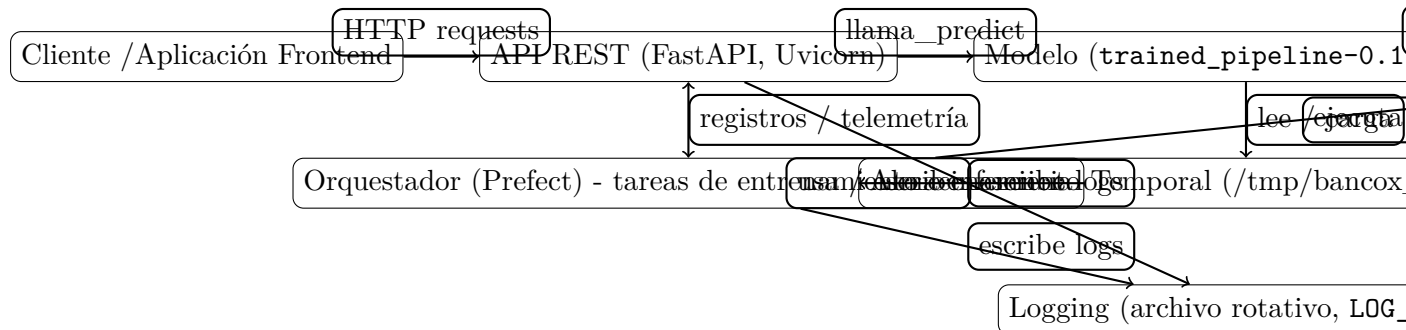


Figure 1: Diagrama de arquitectura: componentes y flujos de datos principales.

5.1. Leyenda y responsabilidades

A continuación se describen las responsabilidades principales de cada componente mostrado en el diagrama:

- **Cliente / Aplicación Frontend:** envía peticiones HTTP al API para solicitar inferencias o consultar el estado del servicio. Implementa validaciones básicas de entrada y consume las respuestas JSON.
- **API REST (FastAPI, Uvicorn):** punto de entrada para solicitudes externas. Valida esquemas (Pydantic), enruta llamadas a funciones de inferencia, aplica manejo

centralizado de errores y expone endpoints de health/readiness. Registra eventos relevantes en el sistema de logging.

- **Modelo (pipeline serializado)**: artefacto serializado que contiene transformaciones y estimador final. Se carga en memoria por la API para realizar predicciones mediante funciones como `predict_pipeline` y `predict_pipeline_proba`. Debe mantenerse versionada y consistente con la lista de features esperadas.
- **Orquestador (Prefect)**: coordina flujos de entrenamiento e inferencia asíncrona, registra telemetría de ejecución y ejecuta tareas programadas (por ejemplo, reentrenamientos). Maneja persistencia intermedia y notificaciones dentro del pipeline.
- **Base de Datos (MySQL)**: almacena las tablas de datos crudos y los datos preparados resultantes del proceso de ETL/training. Es la fuente de verdad para los datasets usados en entrenamientos y auditoría.
- **Almacenamiento Temporal (/tmp, Parquet)**: contenedor temporal para archivos intermedios (parquet) usados durante el preprocesado y reentrenamiento. Facilita transferencia eficiente entre pasos y evita llamadas reiteradas a la DB durante procesamiento.
- **Logging (archivo rotativo, env vars)**: captura eventos, métricas y errores. Permite configurar fichero y nivel mediante variables de entorno (`LOG_FILE`, `LOG_LEVEL`). Prefect y la API escriben en este sistema para auditoría y debugging.

6. LIMITACIONES Y TRABAJO FUTURO

Aunque el enfoque ofrece interpretabilidad, depende de arquitecturas simplificadas. Investigaciones futuras explorarán su extensión a redes recurrentes o convolucionales. [...]

7. TRABAJOS RELACIONADOS

Diversos estudios previos han investigado la extracción de reglas en redes neuronales, incluyendo árboles de decisión sustitutos y métodos de regresión simbólica. [...]

8. CONTRIBUCIONES PRINCIPALES

Este estudio aporta: (1) un algoritmo híbrido simbólico–numérico para extracción de reglas, y (2) una métrica de interpretabilidad basada en similitud del coseno y entropía. [...]

9. CONCLUSIÓN

La combinación de similitud del coseno y razonamiento lógico mejora la explicabilidad de redes neuronales profundas entrenadas, ofreciendo un balance entre interpretabilidad y poder predictivo. [...]

CONFLICTOS DE INTERÉS

Los autores declaran no tener conflictos de interés.

FINANCIAMIENTO

Esta investigación no recibió financiamiento externo.

FECHAS DEL PROCESO

Recibido: Agosto 2025; Aceptado: Agosto 2025; Publicado: Diciembre 2025.

AUTOR CORRESPONDIENTE

Pablo Ariel Negro (pablo.negro@example.com)

REFERENCES

- [1] Rumelhart, D.E., Hinton, G.E., Williams, R.J. (1986). Learning representations by back-propagating errors. *Nature*, 323(6088), 533–536.
- [2] Goodfellow, I., Bengio, Y., Courville, A. (2016). *Deep Learning*. MIT Press.

DOI: 10.4018/IJAIML.347988

Este artículo se publica bajo la licencia de acceso abierto CC-BY 4.0.